

Core Java – Inter-Thread Communication and Deadlock

1. Introduction to Threads

A **thread** is a small unit of a program that can run independently.

For example, while using a computer:

- One task may download a file.
- Another task may play music.
- Another task may display a notification.

These tasks can run at the same time. This is called **multithreading**.

In Java, we can create threads using:

- `Thread` class
- `Runnable` interface

Simple Example

```
class MyThread extends Thread {  
    public void run() {  
        System.out.println("Thread is running");  
    }  
}  
  
class Demo {  
    public static void main(String[] args) {  
        MyThread t = new MyThread();  
        t.start();  
    }  
}
```

Here:

- `MyThread` is a thread.
- `run()` contains the work of the thread.
- `start()` starts the thread.

2. Inter-Thread Communication

Sometimes two or more threads need to **communicate with each other**.

For example, imagine a restaurant:

- The **waiter** takes an order.
- The **chef** prepares the food.
- The waiter should not continuously ask the chef, "Is food ready?"
- The waiter can wait until the chef finishes the food.

Similarly, in Java, one thread may need to **wait** until another thread completes some work.

This is called **Inter-Thread Communication**.

Simple Definition

Inter-thread communication is a mechanism through which one thread communicates or coordinates with another thread.

Java mainly provides three methods:

```
wait()  
notify()  
notifyAll()
```

These methods are available in the `Object` class.

3. Important Methods

3.1 wait()

The `wait()` method makes the current thread **wait**.

The thread releases the lock and waits until another thread sends a notification.

Syntax:

```
wait();
```

Example:

```
synchronized(obj) {  
    obj.wait();  
}
```

In simple words

`wait()` means:

"I will stop for now. Wake me up when the required work is ready."

4. notify()

The `notify()` method wakes up **one waiting thread**.

Syntax:

```
notify();
```

Example:

```
synchronized(obj) {  
    obj.notify();  
}
```

In simple words

`notify()` means:

"The required work is ready. One waiting thread can continue."

5. notifyAll()

The `notifyAll()` method wakes up **all threads waiting on the same object**.

Syntax:

```
notifyAll();
```

Difference

Method	Meaning
<code>wait()</code>	Makes a thread wait
<code>notify()</code>	Wakes one waiting thread
<code>notifyAll()</code>	Wakes all waiting threads

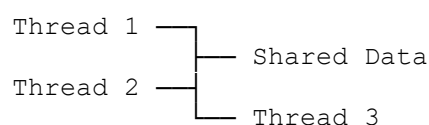
6. Why is synchronized used?

`wait()`, `notify()` and `notifyAll()` are normally used inside a `synchronized` block or method.

Why?

Because multiple threads may try to access the same resource.

For example:



If all threads change the same data at the same time, problems can occur.

`synchronized` helps control access to shared resources.

Simple meaning

`synchronized` means:

"Only one thread should access this critical section at a time."

7. Simple Inter-Thread Communication Example

Consider a producer and consumer.

The **Producer** produces data.

The **Consumer** consumes data.

```
class Data {

    int value;
    boolean available = false;

    synchronized void produce(int value) {

        while (available) {
            try {
                wait();
            } catch (Exception e) {
            }
        }

        this.value = value;
        available = true;

        System.out.println("Produced: " + value);

        notify();
    }

    synchronized void consume() {

        while (!available) {
            try {
                wait();
            } catch (Exception e) {
            }
        }

        System.out.println("Consumed: " + value);

        available = false;

        notify();
    }
}

class Demo {

    public static void main(String[] args) {

        Data data = new Data();

        Thread producer = new Thread(() -> {
            data.produce(100);
```

```
        });  
  
        Thread consumer = new Thread(() -> {  
            data.consume();  
        });  
  
        producer.start();  
        consumer.start();  
    }  
}
```

Possible Output

```
Produced: 100  
Consumed: 100
```

Understanding the Example

The producer puts data into the shared object.

The consumer takes the data.

If data is not available, the consumer waits.

```
Consumer  
  ↓  
Data not available  
  ↓  
wait()  
  ↓  
Producer produces data  
  ↓  
notify()  
  ↓  
Consumer continues
```

This is the basic idea behind inter-thread communication.

8. Real-Life Examples of Inter-Thread Communication

Inter-thread communication is useful in many situations.

Example 1: Online Shopping

One thread downloads product information.

Another thread displays the information.

The display thread may wait until the data is available.

Example 2: Printing

One thread sends documents to a printer.

Another thread manages the printing process.

Example 3: Banking

One thread may update an account.

Another thread may generate a transaction report.

Example 4: Producer-Consumer

One thread produces data.

Another thread consumes the data.

This is one of the most common examples used to understand inter-thread communication.

9. Deadlock

Now consider a situation where two people are waiting for each other.

For example:

```
Person A has Pen  
Person B has Paper
```

```
A wants Paper  
B wants Pen
```

A waits for B.

B waits for A.

Nobody can continue.

This situation is called **Deadlock**.

10. Deadlock in Java

In Java, a deadlock occurs when two or more threads are **waiting for each other forever**.

Simple Definition

Deadlock is a situation where two or more threads are permanently waiting for resources held by each other.

Example:

```
Thread 1  
  ↓  
Has Resource A  
  ↓  
Waiting for Resource B  
  
Thread 2  
  ↓  
Has Resource B  
  ↓  
Waiting for Resource A
```

Neither thread can continue.

11. Simple Deadlock Example

```
class Demo {

    static Object obj1 = new Object();
    static Object obj2 = new Object();

    public static void main(String[] args) {

        Thread t1 = new Thread(() -> {

            synchronized (obj1) {

                System.out.println("Thread 1: Locked obj1");

                try {
                    Thread.sleep(100);
                } catch (Exception e) {
                }

                synchronized (obj2) {
                    System.out.println("Thread 1: Locked obj2");
                }
            }
        });

        Thread t2 = new Thread(() -> {

            synchronized (obj2) {

                System.out.println("Thread 2: Locked obj2");

                try {
                    Thread.sleep(100);
                } catch (Exception e) {
                }

                synchronized (obj1) {
                    System.out.println("Thread 2: Locked obj1");
                }
            }
        });

        t1.start();
        t2.start();
    }
}
```

What happens?

Thread 1 gets:

obj1

Thread 2 gets:

obj2

Now:

Thread 1 → waiting for obj2

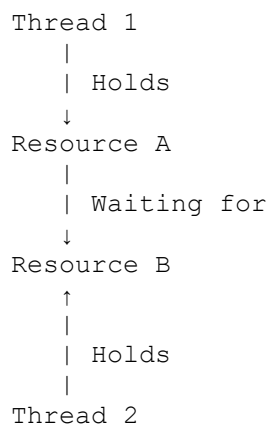
Thread 2 → waiting for obj1

Both threads keep waiting.

Therefore, the program may appear to **hang**.

This is a deadlock.

12. Deadlock Diagram



In simple words:

Thread 1 → has A → wants B

Thread 2 → has B → wants A

Both wait forever.

13. Conditions Required for Deadlock

Deadlock generally occurs when four conditions exist.

1. Mutual Exclusion

Only one thread can use a resource at a time.

Example:

`Only one thread can access the object.`

2. Hold and Wait

A thread is holding one resource while waiting for another.

Example:

`Thread 1 has A
Thread 1 wants B`

3. No Preemption

A resource cannot be forcibly taken away from a thread.

The thread must release it itself.

4. Circular Wait

Threads form a circular waiting relationship.

Example:

`Thread 1 → waiting for Thread 2
Thread 2 → waiting for Thread 1`

These four conditions together can result in deadlock.

14. Deadlock vs Inter-Thread Communication

Inter-Thread Communication	Deadlock
Threads communicate with each other	Threads wait for each other
Uses <code>wait()</code> and <code>notify()</code>	Usually caused by improper locking
Helps coordinate threads	Stops threads from progressing
Can improve program coordination	Can make program hang
<code>synchronized</code> is commonly used	Multiple locks may cause problems

15. How to Avoid Deadlock

Deadlock can be reduced or avoided by following some simple rules.

1. Keep Locking Simple

Do not use unnecessary locks.

2. Lock Resources in the Same Order

For example, always lock:

```
Resource A  
then  
Resource B
```

Do not let another thread lock:

```
Resource B  
then  
Resource A
```

3. Avoid Long Locking

Do not keep a lock for a long time.

4. Avoid Unnecessary Nested Locks

Try not to write:

```
synchronized(a) {  
    synchronized(b) {  
        // work  
    }  
}
```

unless it is really required.

16. Quick Revision

Inter-Thread Communication

It allows threads to communicate and coordinate with each other.

Important methods:

```
wait()  
notify()  
notifyAll()
```

wait()

Makes a thread wait.

notify()

Wakes one waiting thread.

notifyAll()

Wakes all waiting threads.

synchronized

Controls access to shared resources.

Deadlock

A situation where threads wait for each other forever.

Example:

```
Thread 1 → A → waiting for B
```

```
Thread 2 → B → waiting for A
```

17. One-Line Definitions

Thread: A small unit of execution inside a program.

Multithreading: Running multiple threads within a program.

Inter-Thread Communication: Communication or coordination between multiple threads.

wait(): Makes the current thread wait.

notify(): Wakes one waiting thread.

notifyAll(): Wakes all waiting threads.

synchronized: Allows controlled access to shared resources.

Deadlock: A situation where two or more threads wait for each other forever.

18. Easy Way to Remember

`wait()` → "I will wait."

`notify()` → "One thread, wake up!"

`notifyAll()` → "Everyone, wake up!"

`synchronized` → "One at a time."

`Deadlock` → "You wait for me, I wait for you!"

Final Summary

Inter-thread communication is used when threads need to **coordinate their work**. Java provides `wait()`, `notify()` and `notifyAll()` for this purpose.

Deadlock is a **problem** that occurs when threads are waiting for resources held by each other. Proper synchronization, consistent lock ordering and avoiding unnecessary locks can help prevent deadlock.